

1. Overview

This document outlines the SQL Server–based cleanup process implemented to remove error logs, info logs, and other unwanted log data from the PAM Database. The primary objective of this activity is to optimize database performance by ensuring efficient data management and maintaining a clean, lightweight database environment.

2. Purpose

The cleanup script performs the following:

- Clear redundant and temporary log data.
- Deletes older unwanted records from high-transaction tables in batches.
- Removes duplicate entries if any from specific tables.
- Temporarily creates and drops indexes to enhance cleanup performance.
- Ensures smooth execution without downtime or lock contention.

3. Execution Summary

S N.	Action	Description
1	Truncate Temporary Log Tables	Instantly clears log and execution data tables such as error logs, application logs, and API logs.
2	Create Temporary Indexes	Adds indexes to speed up data cleanup operations on high-volume tables.
3	Batch Delete Old Data	Deletes records older than a specific retention period (1 day, 7 days, or 1 hour) in small batches to avoid locking.
4	Remove Duplicates (Table 1)	Cleans up duplicate data from sso_users_services_timebased_command.
5	Remove Duplicates (Table 2)	Cleans up duplicate data from sso_user_restricted_commands.

6	Drop Temporary Indexes	Removes temporary indexes to restore schema to original state after cleanup.
---	-------------------------------	--

4. Detailed Script Explanation

Step 1: Truncate Error and Info Data

The following tables are truncated to remove unwanted or temporary data:

```
TRUNCATE TABLE sso_arcos_applicationlog;
TRUNCATE TABLE ArconPamApiLog;
TRUNCATE TABLE sso_ApiRequestResponseDetails;
TRUNCATE TABLE sso_error_logs;
TRUNCATE TABLE sso_execheck;
TRUNCATE TABLE SIEMARCOSLog;
TRUNCATE TABLE SIEMARCOSCommandLog;
```

Purpose:

These tables hold transient data such as application activity logs, API request/response details, and error traces — which are non-critical after execution and can be cleared regularly.

Step 2: Create Temporary Indexes for Performance

```
CREATE INDEX idx_tmp_SIEMARCOSenvision_EventTimeStamp
ON SIEMARCOSenvision (EventTimeStamp);
-- (Similar index creation on all major cleanup tables)
```

Purpose:

Temporary indexes are added to speed up batch deletion queries that filter by date or key columns.

Note:

These indexes are dropped after cleanup to prevent schema clutter.

Step 3: Batch Delete Old Data

Old records are deleted in controlled batches using a loop mechanism:

Example:

```
WHILE 1 = 1
BEGIN
    DELETE TOP (10000)
    FROM SIEMARCOSenvision
    WHERE EventTimeStamp < DATEADD(DAY, -1, GETDATE());

    IF @@ROWCOUNT = 0 BREAK;
    WAITFOR DELAY '00:00:01';
END;
```

Tables & Retention Policies:

Table Name	Column Used	Retention Period	Deletion Logic
SIEMARCOSenvision	EventTime Stamp	1 Day	Delete old event logs
sso_service_log_realtime	slr_start ed_on	7 Days	Delete old service runtime logs
sso_arcos_out_box	aob_creat ed_on	1 Day	Delete processed outbound records
sso_Api_User_Log	LogInDate Time	1 Hour	Delete stale session logs

Batch Strategy:

- Deletes **10,000 records** per iteration.
- Waits **1 second** after each batch.
- Reduces lock escalation and transaction log growth.

Step 4: Remove Duplicates from sso_users_services_timebased_command

```
;WITH CTE_Duplicates AS (  
    SELECT stc_id,  
           ROW_NUMBER() OVER (PARTITION BY stc_ust_id, stc_ssc_id  
ORDER BY stc_id) AS rn  
    FROM sso_users_services_timebased_command  
)  
DELETE TOP (10000)  
FROM sso_users_services_timebased_command  
WHERE stc_id IN (  
    SELECT stc_id FROM CTE_Duplicates WHERE rn > 1  
);
```

Purpose:

Removes duplicate service-time-based command mappings while retaining the smallest stc_id.

Step 5: Remove Duplicates from sso_user_restricted_commands

```
;WITH CTE_Duplicates AS (  
    SELECT surc_id,  
           ROW_NUMBER() OVER (PARTITION BY sus_service_id, sus_sud_id,  
ssc_id ORDER BY surc_id) AS rn  
    FROM sso_user_restricted_commands  
)  
DELETE TOP (10000)  
FROM sso_user_restricted_commands  
WHERE surc_id IN (  
    SELECT surc_id FROM CTE_Duplicates WHERE rn > 1  
);
```

Purpose:

Cleans duplicate user restriction entries to maintain data integrity.

Step 6: Drop Temporary Indexes

```
DROP INDEX idx_tmp_SIEMARCOsenvision_EventTimeStamp ON
SIEMARCOsenvision;
-- (Similar drop statements for all temporary indexes)
```

Purpose:
Removes all temporary indexes created for cleanup performance.

5. Performance and Safety Measures

Parameter	Description
Batch Size	10,000 rows per batch ensures minimal locking impact.
Delay Between Batches	1 second (WAITFOR DELAY '00:00:01') gives system breathing room.
Transaction Logging	Each batch is atomic, avoiding large log growth.
Temporary Indexing	Indexes only exist during cleanup for faster WHERE lookups.
No Stored Procedures	Direct execution via SQL script ensures compatibility in all environments.

6. Execution Recommendations

Recommendation	Description
Run During Low Traffic Hours	Execute during maintenance windows (e.g., midnight).
Backup Before Execution	Always take full DB backup before cleanup.
Monitor Log Size	Keep an eye on transaction logs during first few runs.

7. Expected Output

Upon successful execution:

- Old data and duplicates are removed.
- Temporary indexes are dropped.
- Console message displays:

```
PRINT 'Data cleanup completed successfully!';
```

8. Rollback and Validation

Rollback:

Not applicable for TRUNCATE and DELETE operations. Ensure prior **full database backup** is available for restoration.

Validation Queries:

```
SELECT COUNT(*) FROM sso_error_logs;  
SELECT COUNT(*) FROM sso_Api_User_Log WHERE LogInDateTime <  
DATEADD(HOUR, -1, GETDATE());
```

Confirm reduced record counts and performance improvement post-cleanup.